

**[SOFTWARE EDIFICIO MIRADOR]
(DAS) Documento Arquitectura de Software
Versión 3.0.1**

Identificación de Documento

Identificación	EM1
Proyecto	SOFTWARE EDIFICIO MIRADOR
Versión	3.0.1
Documento mantenido por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última revisión	31/03/2026
Fecha de próxima revisión	02/04/2026
Documento aprobado por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última aprobación	21-06-2026

Historia de Revisiones

Fecha	Versión	Descripción	Autor
04/05/2026	1.0.0	Línea Base Inicial: Creación oficial del documento DAS. Estructuración del Ítem 1 (Introducción) definiendo el contexto del problema, propósito inicial y el alcance del sistema del Edificio Mirador.	C. Betancur, E. Tapia, Á. Salazar
06/05/2026	1.0.1	Parche de Redacción: Ajustes y correcciones menores en la descripción del Contexto del Problema. Incorporación de la tabla vacía de acrónimos en la sección 1.4.	C. Betancur, E. Tapia, Á. Salazar
08/05/2026	1.0.2	Ajuste de Formato: Corrección en el formato del índice y maquetación de los marcadores de posición para el inicio del Ítem 2.	C. Betancur, E. Tapia, Á. Salazar
12/05/2026	1.1.0	Actualización de Visión: Expansión del documento al Ítem 2 (Visión del Sistema). Redacción completa de la descripción general de la plataforma y definición de los objetivos estratégicos (2.1 y 2.2).	C. Betancur, E. Tapia, Á. Salazar
15/05/2026	1.2.0	Adición de Requerimientos: Incorporación y desglose de los Requerimientos Funcionales principales (RF-1 al RF-6) en la sección 2.3.	C. Betancur, E. Tapia, Á. Salazar
18/05/2026	1.2.1	Parche de Atributos: Adición de los Requerimientos No Funcionales iniciales cubriendo criterios base de Usabilidad, Rendimiento, Fiabilidad y Seguridad (2.4).	C. Betancur, E. Tapia, Á. Salazar
20/05/2026	1.2.2	Parche de Entorno: Redacción de la sección de Supuestos y Dependencias (2.5), detallando las necesidades de conectividad y capacitación de la administración.	C. Betancur, E. Tapia, Á. Salazar

22/05/2026	1.2.3	Cierre de Fase de Requerimientos: Revisión integral de consistencia de los ítems 1 y 2. Pulido de la redacción de los flujos funcionales y congelamiento de la primera gran etapa del DAS.	C. Betancur, E. Tapia, Á. Salazar
22/05/2026	2.0.0	Cambio Mayor - Arquitectura: Incorporación del Ítem 3 (Estilos y Patrones Arquitectónicos). Justificación técnica de la adopción del patrón de capas MVC y el desacoplamiento mediante componentes DAO.	C. Betancur, E. Tapia, Á. Salazar
02/06/2026	2.1.0	Modelado de Vistas (4+1): Inserción masiva de diagramas UML en el Ítem 4. Inclusión de diagramas de Casos de Uso (General y Subsistema 3), Diagrama de Clases, Diagramas de Actividades, Componentes, Paquetes y Despliegue. Redacción del primer borrador del CU-003.	C. Betancur, E. Tapia, Á. Salazar
08/06/2026	2.1.1	Métricas de Calidad: Redacción de la Sección 5 (Requisitos de Calidad). Formalización de la tabla de atributos de calidad, definición de criterios de aceptación y métodos de evaluación para el motor financiero.	C. Betancur, E. Tapia, Á. Salazar
18/06/2026	3.0.0	Cambio Mayor - Validación e Interfaz: Versión correspondiente al archivo actual DAS-3.0.0.docx. Incorporación de los Principios de Diseño (Sección 6) y Mockups de alta fidelidad exportados de Figma (Sección 7). Integración del análisis detallado de la Evaluación Heurística de Nielsen (Sección 8), justificación del Control de Versiones (Sección 9), Conclusiones, Bibliografía y Anexo de Carta Gantt.	C. Betancur, E. Tapia, Á. Salazar
21/06/2026	3.0.1	Corrección de Hallazgos (Planificada): Resolución de las inconsistencias detectadas en la v3.0.0. Alineación de los IDs de Casos de Uso, eliminación del diagrama de componentes duplicado en la introducción, reordenamiento secuencial de los capítulos de cierre (10, 11 y 12) y completado del glosario de acrónimos.	C. Betancur, E. Tapia, Á. Salazar

Tabla de Contenidos

1. INTRODUCCIÓN.....	5
1.1. CONTEXTO DEL PROBLEMA	5
1.2. PROPÓSITO	5
1.3. ÁMBITO	5
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	5
1.5. RESUMEN EJECUTIVO	6
1.6. ARQUITECTURA DEL SISTEMA	6
2. VISIÓN DEL SISTEMA	11
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	11
2.2. OBJETIVOS DEL SISTEMA.....	11
2.3. REQUERIMIENTOS FUNCIONALES	11
2.4. REQUERIMIENTOS NO FUNCIONALES	12
2.5. SUPUESTOS Y DEPENDENCIAS.....	12
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	13
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	13
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS.....	14
4.1. VISTA DE ESCENARIO.....	14
4.1.1. <i>Propósito</i>	14
4.1.2. <i>Actores</i>	14
4.1.3. <i>Diagrama general de casos de uso</i>	15
4.1.4. <i>Diagrama de casos de uso específicos</i>	15
4.1.6. <i>Especificación de casos de uso</i>	16
4.2. VISTA LÓGICA.....	17
4.2.1. <i>Propósito</i>	17
4.2.2. <i>Diagrama de clases</i>	18
4.2.3. <i>Descripción diagrama de clases</i>	18
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	19
4.3.1. <i>Propósito</i>	19
4.3.2. <i>Diagrama de componente</i>	20
4.3.3. <i>Descripción diagrama de componente</i>	20
4.3.4. <i>Diagrama de paquete</i>	20
4.3.5. <i>Descripción diagrama de paquete</i>	21
4.4. VISTA DE PROCESOS	21
4.4.1. PROPÓSITO	21
4.4.2. DIAGRAMA DE ACTIVIDAD	22
4.4.3. DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD	22
4.5. VISTA FÍSICA.....	23
4.5.1. <i>Propósito</i>	23
4.5.2. <i>Diagrama de despliegue</i>	23
4.5.3. <i>Descripción diagrama de despliegue</i>	23
5. REQUISITOS DE CALIDAD	24

5.1. PROPÓSITO	24
5.3. <i>Reglas y criterios de evaluación de calidad</i>	25
6. PRINCIPIOS DE DISEÑO APLICADOS	26
6.1. <i>Propósito</i>	26
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	26
7. PROTOTIPO	27
7.1. PROPÓSITO	27
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	28
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	31
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	31
8.1. PROPÓSITO	31
8.2. LISTA DE VERIFICACIÓN	32
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	38
9. CONTROL DE VERSIONES	41
9.1. PROPÓSITO	41
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	41
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	41
7. CONCLUSIONES	42
8. BIBLIOGRAFÍA	42
9. ANEXOS	43
9.1. CARTA GANTT	43

1. INTRODUCCIÓN

1.1. Contexto del Problema

El problema se sitúa en que una comunidad de gran tamaño, 'Edificio Mirador', depende actualmente de un sistema de gestión obsoleto. Esta situación genera una serie de dificultades, entre las que destacan la falta de confianza por parte de los residentes, la ineficiencia en los procesos de cobro y un aumento en los niveles de morosidad, debido principalmente a la inexistencia de un mecanismo eficaz para identificar y monitorear oportunamente a quienes cumplen con sus pagos y a quienes presentan atrasos.

La raíz del problema radica en el **procesamiento manual de datos y la falta de una base de datos centralizada**, lo que impide la integridad de la información y genera cuellos de botella en la validación de cobros. Técnicamente, la inexistencia de algoritmos de cálculo automatizado para intereses y la carencia de módulos de notificación en tiempo real provocan errores humanos recurrentes, imposibilitando una gestión financiera transparente y escalable para una comunidad de gran tamaño.

1.2. Propósito

Implementar un sistema de gestión moderno y automatizado que permita mejorar el control administrativo y financiero del edificio, facilitando la toma de decisiones, el seguimiento de pagos y la gestión eficiente de la información.

1.3. Ámbito

El proyecto se centra en modernizar la administración del edificio "El Mirador", optimizando la determinación, cálculo automatizado y notificación de los montos correspondientes a gastos comunes, disminuyendo la morosidad, agilizando procesos internos y fortaleciendo la comunicación con los residentes a través del seguimiento de solicitudes y reclamos, asegurando un funcionamiento más eficiente y transparente.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION
<i>Prorratio</i>	Algoritmo financiero encargado de distribuir proporcionalmente la suma total de los gastos comunes del edificio entre todos los departamentos y bodegas, utilizando el porcentaje de alícuota de cada uno.
<i>RF</i>	Requerimiento Funcional. Declaración de un servicio o función que el sistema debe proveer de manera obligatoria (por ejemplo, calcular el gasto común o emitir un aviso de cobro).
<i>UML</i>	<i>Unified Modeling Language</i> (Lenguaje Unificado de Modelado). Lenguaje estándar visual que se utiliza en este documento para especificar, visualizar y documentar los artefactos del sistema (diagramas de clases, componentes, despliegue, etc.).

<i>Mockup</i>	Representación visual o maqueta estática a escala de la interfaz de usuario del sistema, que sirve para validar el diseño gráfico y la disposición de elementos antes de programar.
<i>Evaluación Heurística</i>	Método de inspección de usabilidad para sistemas de software basado en los 10 principios de diseño de Jakob Nielsen, utilizado para identificar problemas en la interfaz de usuario.

1.5. Resumen ejecutivo

El Edificio Mirador padece una crisis de morosidad y desconfianza causada por un sistema de gestión manual y obsoleto que impide la integridad de los datos. Se propone implementar una solución tecnológica automatizada bajo una arquitectura MVC, diseñada para centralizar la información y eliminar errores humanos en el cálculo de deudas.

El sistema define dos perfiles críticos: el Administrador, responsable de la gestión financiera y reportes de cobranza, y el Residente, quien podrá consultar estados de cuenta y gestionar reclamos en tiempo real. Con esta modernización, se garantiza la trazabilidad total de los fondos, una comunicación interna fluida y una reducción drástica de la morosidad mediante notificaciones automáticas.

1.6. Arquitectura del sistema

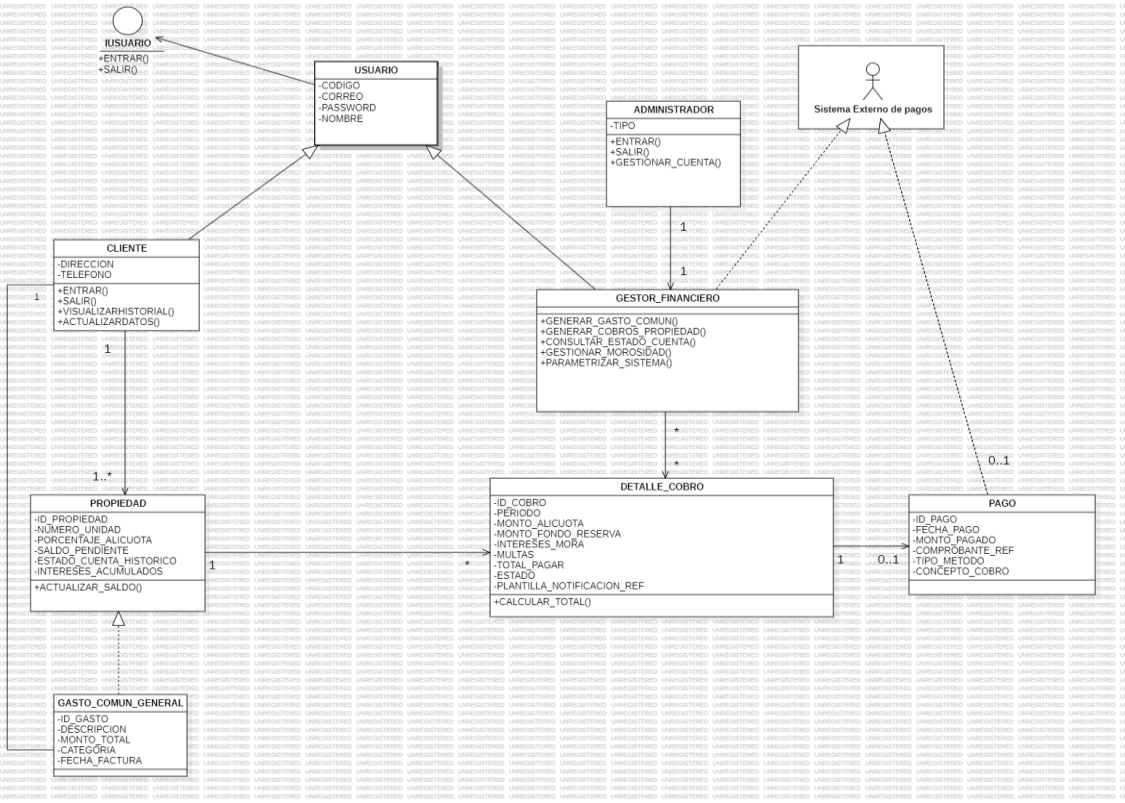
(ej. vista de escenario, vista lógica, vista de desarrollo, vista de proceso, vista física)

VISTA ESCENARIO

-Diagrama Casos de uso general, todo el proyecto

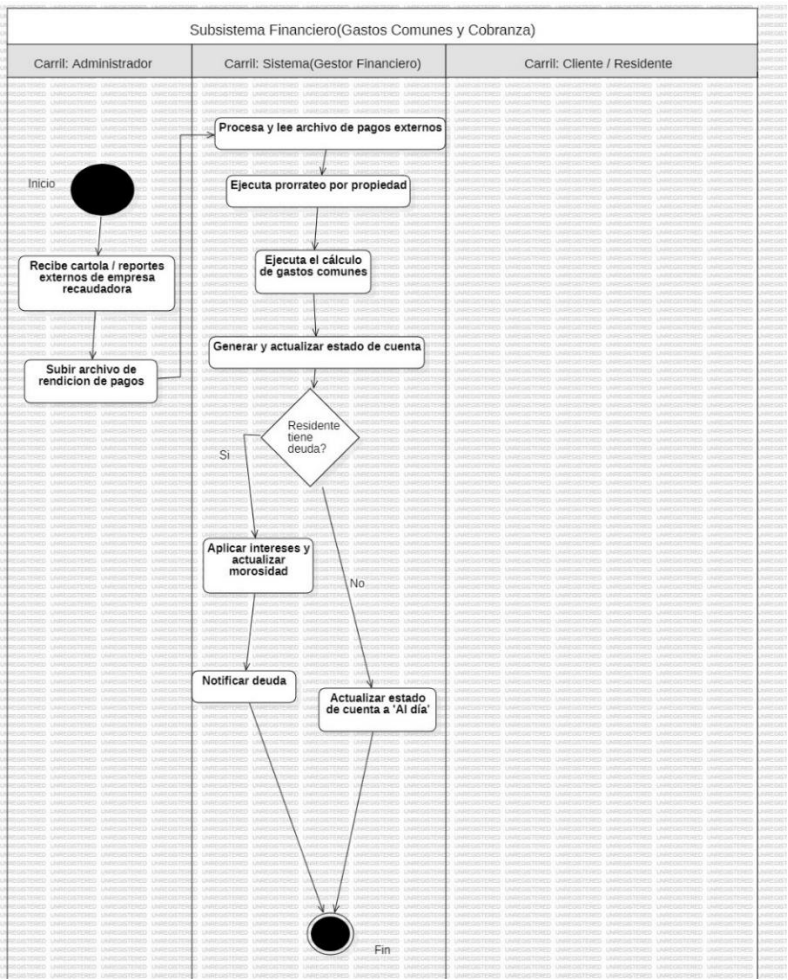
VISTA LOGICA

-Diagrama de clases



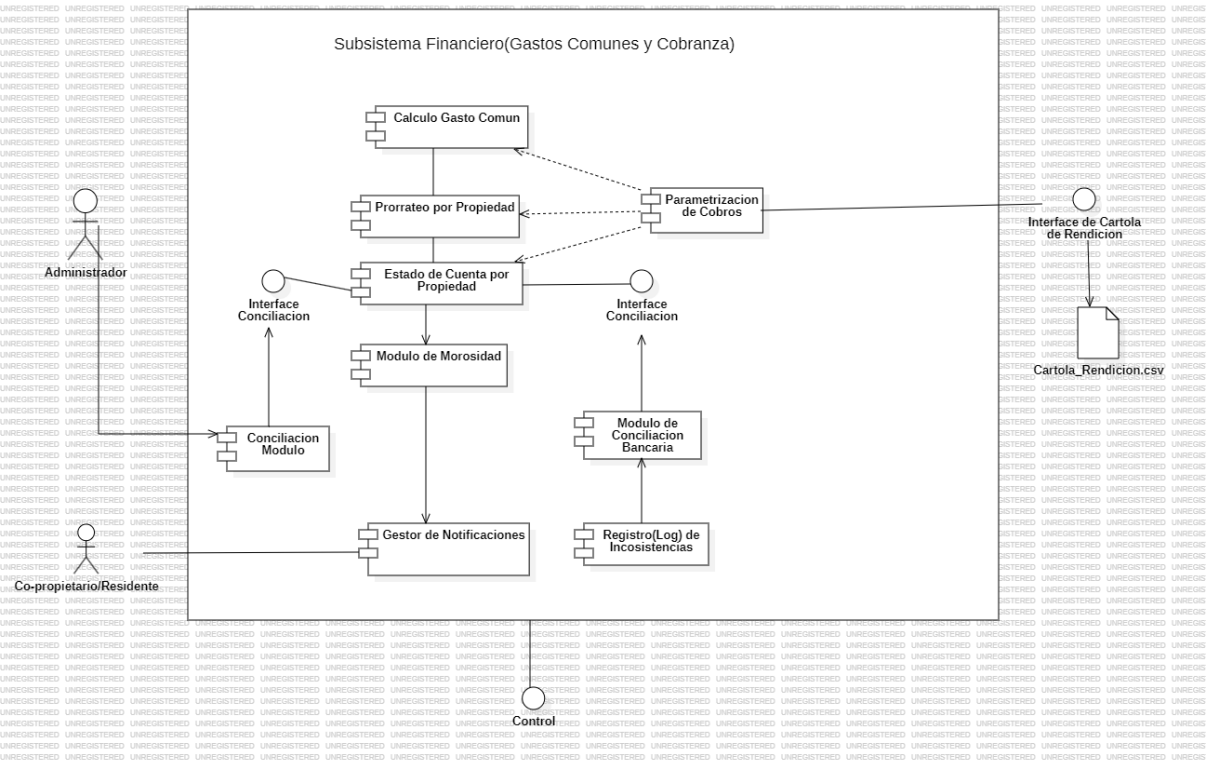
VISTA DE PROCESOS

-Diagrama de actividades

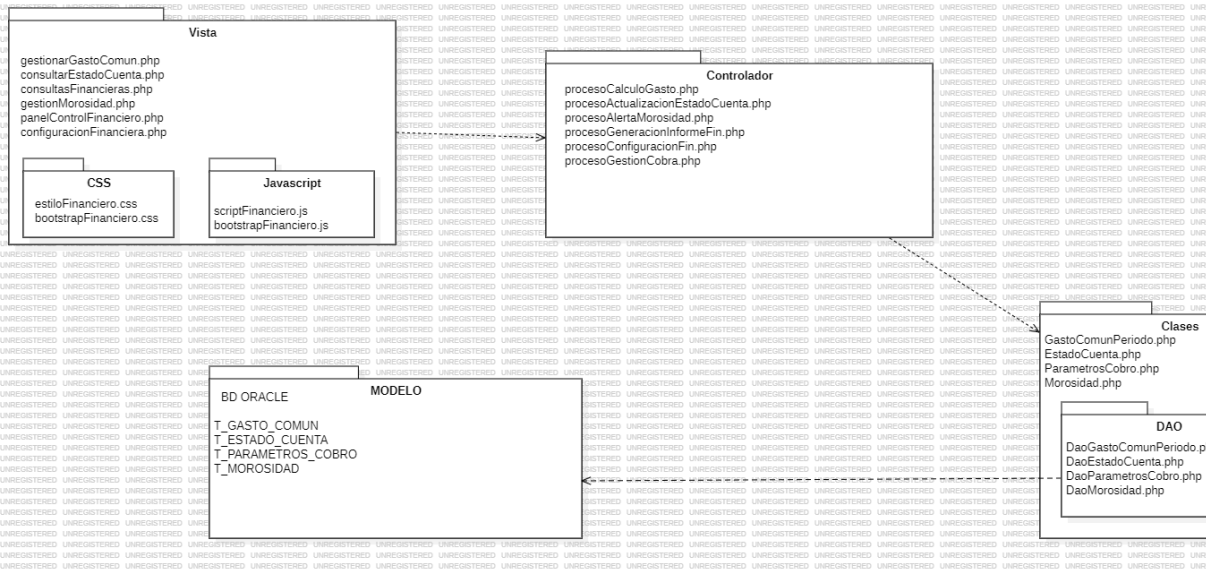


VISTA DESARROLLO

-Diagrama de componentes

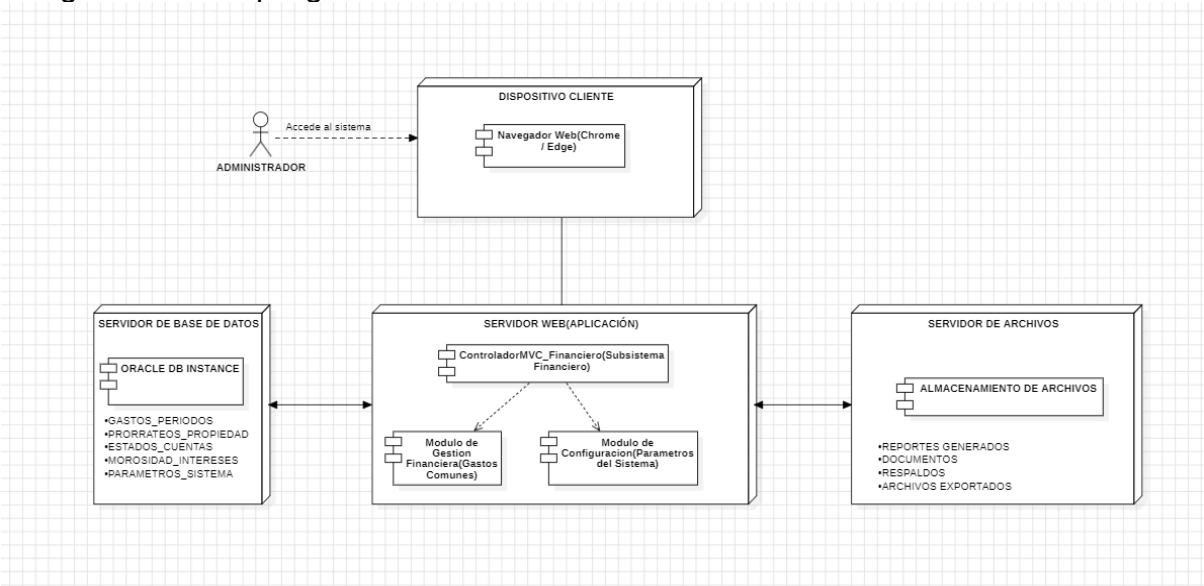


-Diagrama de paquetes



VISTA FÍSICA

-Diagrama de despliegue



2. VISIÓN DEL SISTEMA

2.1.Descripción general del sistema

El sistema es una plataforma de gestión que permitirá administrar residentes, registrar pagos de gastos comunes, controlar la morosidad y generar reportes financieros, la organización y transparencia en la administración del edificio Mirador.

2.2.Objetivos del sistema

Desarrollar e implementar una solución tecnológica que permita automatizar el cálculo del gasto común, el prorrateo por propiedad y el control de morosidad del edificio "Mirador", integrando la carga de datos externos de recaudación para asegurar la trazabilidad del estado financiero de cada departamento.

2.3.Requerimientos funcionales

ID	Nombre Requerimiento	Descripción	Actor
RF-1	Gestión de residentes	Registrar, actualizar y eliminar información de residentes, asociándolos a departamentos específicos	Administrador
RF-2	Gestión de pagos de gastos comunes	Registrar pagos realizados por los residentes, Generar estados de cuenta	Administrador

		individuales, Calcular automáticamente deudas y morosidad. Emitir comprobantes de pago	
RF-3	Gestión de Solicitudes y Reclamos	Un módulo para que el residente ingrese quejas y la administración responda	Sistema
RF-4	Sistema de Notificaciones	Automatización de correos electrónicos para avisos de cobro y alertas de morosidad	Residente / Administrador
RF-5	Control de morosidad	Identificar residentes con pagos pendientes. Generar alertas y notificaciones automáticas. Aplicar intereses o recargos por atraso (si corresponde)	Sistema
RF-6	Generación de informes	Generar reportes financieros (ingresos, deudas, morosidad).	Administrador

2.4. Requerimientos no funcionales

Usabilidad: El sistema debe ser intuitivo tanto para los residentes como para la administración del edificio.

Rendimiento y Disponibilidad: El sistema debe permitir un acceso transparente y a tiempo real de la información dentro del sistema (pagos, etc). Se sugiere que el tiempo de carga del sistema sea inferior a 2 segundos.

Fiabilidad: El sistema debe asegurar precisión en la cobranza y eficiencia de la gestión de recursos para garantizar la transparencia financiera.

Seguridad: El sistema debe garantizar la protección de los datos financieros tanto de los residentes como de la administración.

2.5. Supuestos y dependencias

Acceso a tecnología: La administración deberá contar con un computador y acceso a internet para poder actualizar los datos de pago y cobro en tiempo real.

Información de empresa: Se necesita la información de los servicios de luz, agua, etc. para poder ingresar las cifras al sistema y poder gestionar los pagos de manera automática

Capacitación: El sistema depende de que el personal esté dispuesto a capacitarse para usar el software de la manera indicada.

Comunicación: La administración es la encargada de gestionar los datos, la información se le hará llegar a los residentes directamente a su departamento o por correo electrónico si así lo desean.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado (ej. monolítico, microservicios, SOA, capas)

Para el desarrollo del Subsistema Financiero (Gastos Comunes y Cobranza) se utilizó una arquitectura en capas basada en el patrón MVC y componentes DAO.

- Capa de Presentación (Vista): Interfaces en PHP, CSS y JavaScript donde el administrador ingresa los gastos comunes (RF-08) y los residentes consultan su estado de cuenta (RF-12).
- Capa de Lógica de Negocio (Controlador): Procesa los cálculos del sistema, como el prorrateo por propiedad (RF-09), la detección de morosidad e intereses (RF-13) y las reglas de cobro configurables (RF-28).
- Capa de Acceso a Datos (Modelo / DAO): Contiene las clases del sistema y los DAO que realizan consultas SQL seguras.
- Capa de Persistencia: Base de datos Oracle, donde se almacenan las tablas financieras del sistema.

Esta arquitectura permite organizar el sistema, mejorar la seguridad de los datos y facilitar el mantenimiento.

3.2. Justificación del estilo según el contexto del sistema

Se eligió una **arquitectura en capas basada en el patrón MVC** porque permite organizar de mejor forma el **Subsistema Financiero**, encargado del cálculo y gestión de los gastos comunes.

Este estilo facilita la **mantenibilidad y escalabilidad del sistema**, permitiendo manejar correctamente procesos como la **generación del gasto común (RF-08)**, el **prorrateo por propiedad (RF-09)**, la **consulta de estado de cuenta (RF-12)** y la **gestión de morosidad e intereses (RF-13)**.

Además, permite **aislar la base de datos**, asegurando la integridad y seguridad de la información financiera configurada mediante la **parametrización de cobros (RF-28)**.

3.3. Patrones de diseño aplicados (ej. patrón MVC, repositorio, etc.)

Patrón MVC (Modelo – Vista – Controlador)

Este patrón permite separar las responsabilidades dentro del Subsistema Financiero:

- La Vista (PHP) muestra las interfaces donde el administrador ingresa los gastos comunes y los residentes consultan su estado de cuenta.
- El Controlador procesa la lógica del sistema, como el cálculo del prorrateo, la

generación de cargos y la detección de morosidad.

- El Modelo gestiona las entidades financieras del sistema, como gastos comunes, estados de cuenta y pagos.

Patrón DAO

El patrón DAO centraliza todas las consultas SQL hacia la base de datos Oracle, utilizadas para almacenar y recuperar información del subsistema financiero.

Esto evita que la lógica de negocio acceda directamente a las tablas, lo que mejora la seguridad, organización y mantenimiento del sistema, especialmente en operaciones relacionadas con gastos comunes, pagos y deudas.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El propósito de la vista de escenario es definir el alcance funcional del sistema desde la perspectiva de los usuarios finales. Sirve para identificar qué actores interactúan con la plataforma y qué acciones (casos de uso) pueden realizar, validando que se cubran todas las necesidades de administración y finanzas de la comunidad.

4.1.2. Actores

Administrador de la comunidad

Usuario responsable de la gestión financiera del sistema. Puede **generar el gasto común del período (RF-08)**, ejecutar el **prorrateo por propiedad (RF-09)**, configurar reglas de cobro (**RF-28**) y gestionar procesos de cobranza y morosidad (**RF-13**).

Propietario / Arrendatario

Usuarios residentes que utilizan el sistema para **consultar su estado de cuenta (RF-12)**, revisar cargos, pagos y deudas asociadas a su propiedad.

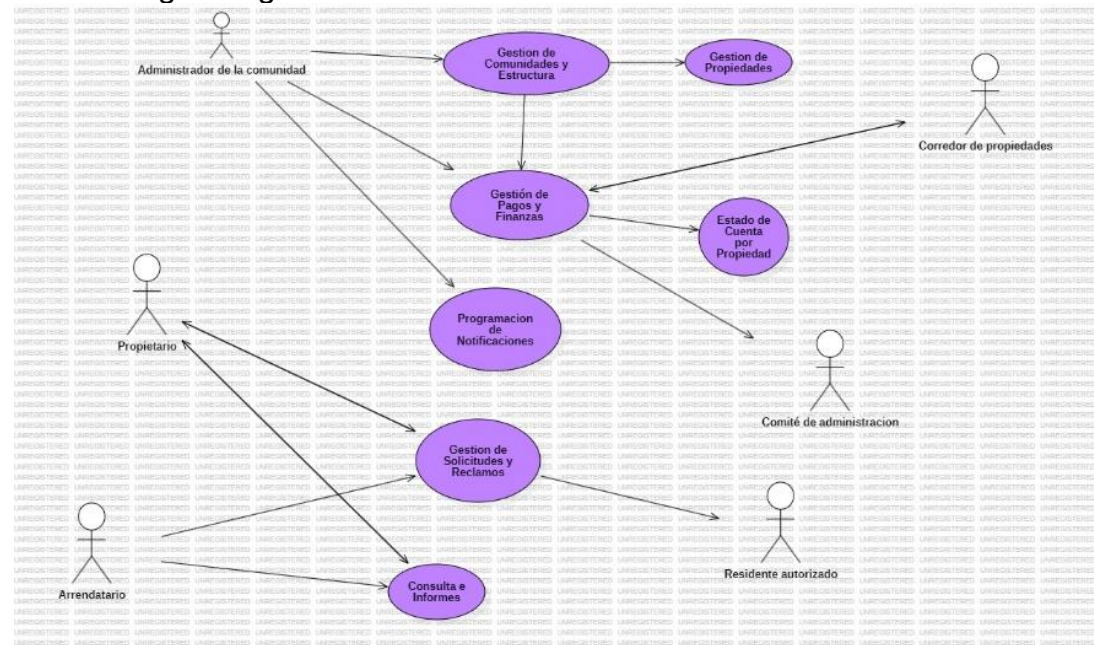
Comité de administración

Grupo supervisor que puede acceder a **información financiera e informes de gestión**, con el objetivo de supervisar el manejo de los gastos comunes y la situación de morosidad de la comunidad.

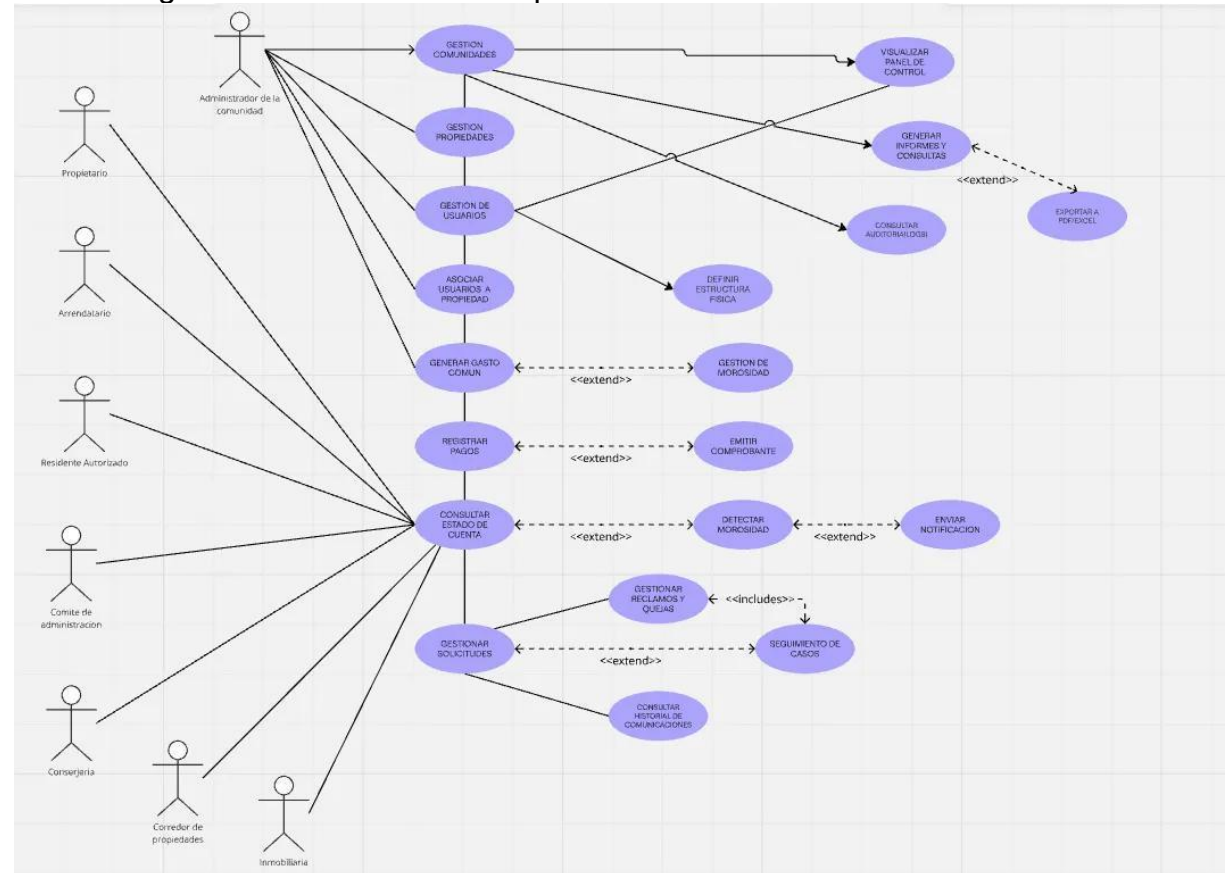
Sistemas externos de pago

Plataformas externas de pago electrónico que permiten **registrar y validar transacciones financieras** realizadas por los residentes, facilitando el proceso de recaudación del gasto común.

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso específicos



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001	Registrar Gastos del Período	Administrador
CU-002	Calcular Gasto Común y Prorratio por Propiedad	Sistema (Gestor Financiero)
CU-003	Generar y Enviar Avisos de Cobro	Sistema
CU-004	Consultar Estado de Cuenta Individual	Propietario / Residente
CU-005	Registrar o Procesar Pago de Gasto Común	Propietario / Sistema Externo de Pagos
CU-006	Detectar Morosidad y Aplicar Intereses	Sistema (Gestor Financiero)
CU-007	Registrar Pago Manual	Administrador
CU-008	Generar Estado de Cuenta por Propiedad	Sistema
CU-009	Generar Informes de Recaudación y Morosidad	Administrador

4.1.6. Especificación de casos de uso

Caso de Uso	Calcular Gasto Común y Prorratio	Identificador: CU-002
Actores	Administrador, Sistema (Gestor Financiero).	
Tipo	Primario	
Referencias	RF-08 (Generación de gasto común), RF-09 (Generación de gasto común por propiedad), RF-12 (Estado de cuenta por propiedad)	
Precondición	El administrador ha registrado en el sistema los gastos del período correspondientes a la comunidad.	
Postcondición	El sistema calcula automáticamente el gasto común y genera los estados de cuenta individuales para cada propiedad , aplicando el prorratio correspondiente.	
Descripción	Este caso de uso permite calcular automáticamente el monto de gasto común que corresponde a cada propiedad, distribuyendo proporcionalmente los gastos registrados del período según los criterios de prorratio definidos por la administración..	
Resumen	Automatiza el cálculo y distribución del gasto común para cada propiedad, reduciendo errores manuales y asegurando transparencia en la gestión financiera de la comunidad.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Propietario / Residente	Ingresa al Panel de Control Financiero y selecciona la opción " Calcular Gasto Común ".
2	Sistema	Recupera y consolida los gastos registrados del período correspondientes a la comunidad (RF-08).
3	Sistema	Ejecuta el proceso de prorrateo por propiedad, distribuyendo los costos según los criterios definidos en el sistema (RF-09).
4	Sistema	Consulta la base de datos para actualizar los estados de cuenta de cada propiedad , incorporando los nuevos cargos del período (RF-12).
5	Sistema	Verifica si existen deudas pendientes o propiedades morosas y calcula los intereses correspondientes según la configuración del sistema (RF-13 y RF-28).
6	Sistema	Genera el estado de cuenta individual para cada propiedad y lo deja disponible para su consulta en la plataforma.

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
2a	Gastos del período no registrados: Si el administrador no ha ingresado los gastos del período correspondientes a la comunidad, el sistema bloquea la ejecución del cálculo de gasto común (RF-08), muestra un mensaje de alerta indicando " <i>No existen gastos registrados para el período actual</i> " y retorna al panel de control financiero.
4a	Propiedad con morosidad previa: Si una propiedad presenta deudas de períodos anteriores, el sistema identifica automáticamente la condición de morosidad y aplica los intereses o recargos configurados según las reglas del sistema (RF-13 y RF-28), incorporándolos al estado de cuenta actualizado de la propiedad.

4.2. VISTA LÓGICA

4.2.1. Propósito

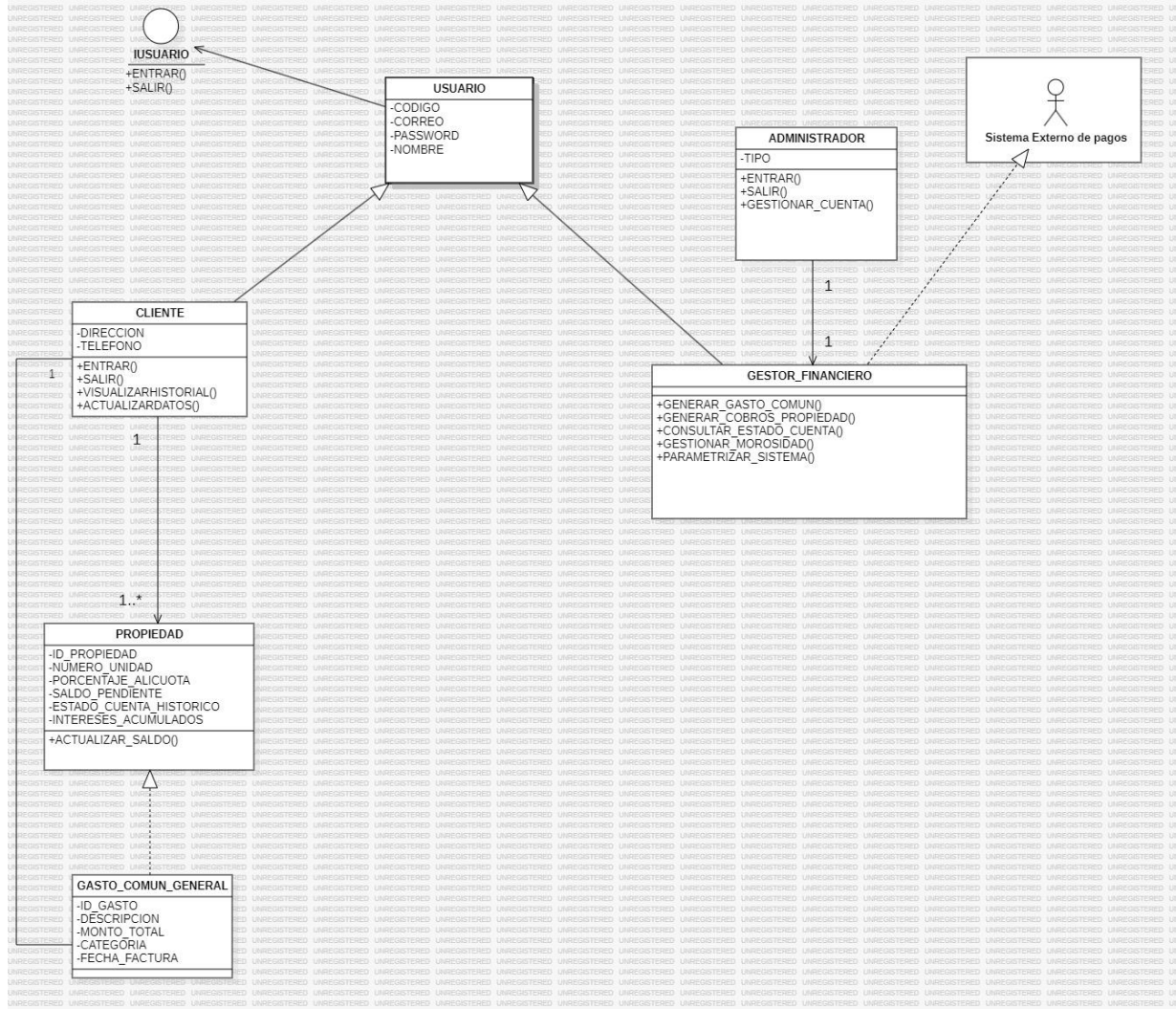
La **vista lógica** describe la estructura interna del **Subsistema Financiero**, mostrando las clases principales del sistema, sus atributos, métodos y las relaciones entre ellas para implementar las reglas de negocio.

Esta vista permite representar cómo el sistema gestiona procesos como la **generación del gasto común del período (RF-08)**, el **prorrateo del gasto común por propiedad (RF-09)**, la **consulta y generación del estado de cuenta por propiedad (RF-12)**, la **identificación de morosidad y cálculo de intereses por**

atraso (RF-13) y la parametrización de las reglas de cobro e intereses del sistema (RF-28).

De esta forma se define la organización lógica de las clases que permiten implementar las funcionalidades financieras del sistema de administración de comunidades.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

Se aplica herencia, donde la clase base **USUARIO** contiene los datos generales del sistema. De ella derivan **PROPIETARIO/RESIDENTE** y **ADMINISTRADOR**, quienes interactúan con el sistema para consultar estados de cuenta o gestionar los procesos financieros. Ambos utilizan la interfaz

IUSUARIO para realizar acciones como autenticarse y acceder a la plataforma.

Gestor Financiero (GESTORFINANCIERO):

Es el componente central del subsistema. Se encarga de ejecutar la lógica de negocio relacionada con la generación del gasto común del período (RF-08), el prorrateo por propiedad (RF-09), la identificación de morosidad y cálculo de intereses (RF-13) y la aplicación de las reglas de cobro configuradas (RF-28).

Gestión de Estados de Cuenta (ESTADOCUENTA):

Representa el registro financiero asociado a cada propiedad, donde se almacenan los cargos, pagos y saldos pendientes. Permite la consulta y generación del estado de cuenta individual (RF-12) para cada propietario o residente.

Sistema Externo de Pagos:

Representa la integración con plataformas externas de pago electrónico, encargadas de procesar y validar las transacciones financieras realizadas por los residentes, permitiendo registrar los pagos y actualizar los estados de cuenta dentro del sistema.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

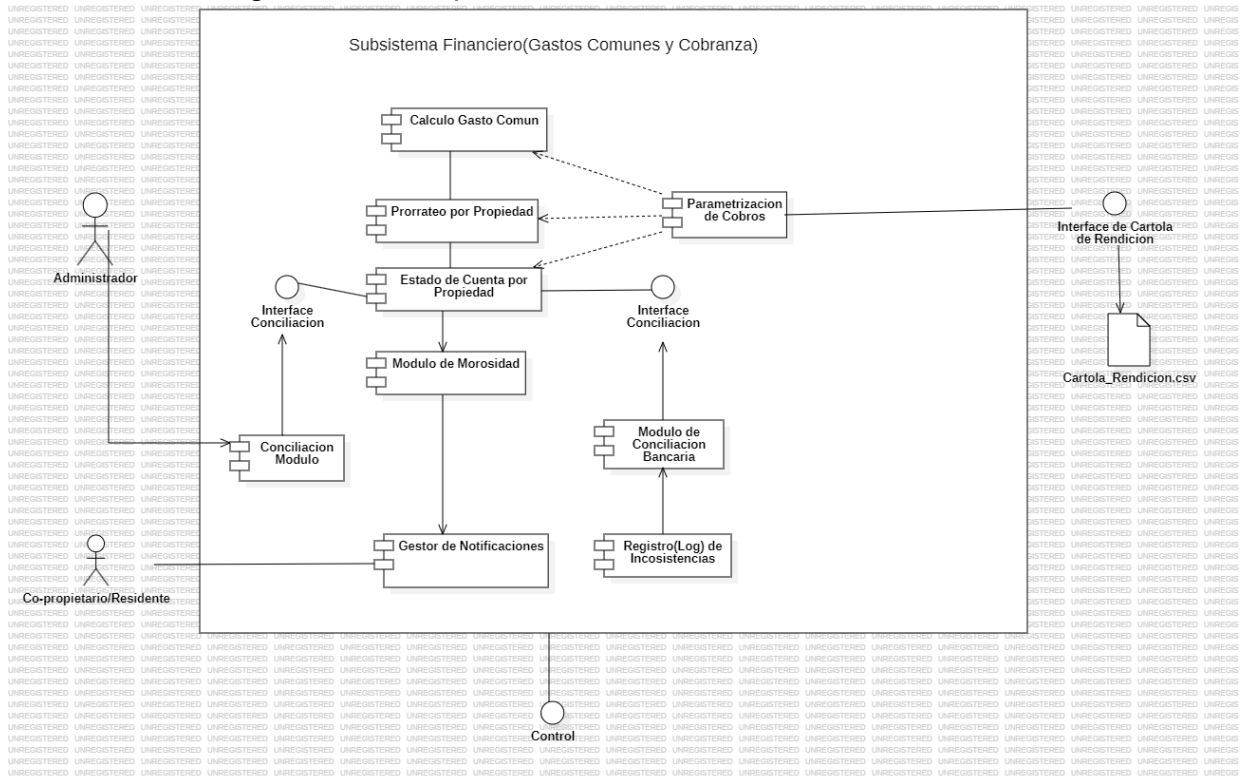
4.3.1. Propósito

El propósito del diagrama de componentes es representar los módulos de software que conforman el Subsistema Financiero y las relaciones o dependencias entre ellos.

Este diagrama permite visualizar cómo se organizan los componentes que implementan funcionalidades como la **generación del gasto común (RF-08)**, el **prorrateo por propiedad (RF-09)**, la **consulta de estados de cuenta (RF-12)**, la **gestión de morosidad e intereses (RF-13)** y la **parametrización de reglas de cobro (RF-28)**.

De esta manera se muestra cómo interactúan los distintos componentes del sistema, como la **interfaz web**, **los controladores financieros**, **la lógica de negocio y el acceso a datos**, garantizando una arquitectura modular y desacoplada.

4.3.2. Diagrama de componente



4.3.3. Descripción diagrama de componente

El diagrama de componentes del **Subsistema 3 (Gestión Financiera: Gastos Comunes y Cobranza)** muestra cómo se organizan los módulos encargados de administrar los procesos financieros del sistema.

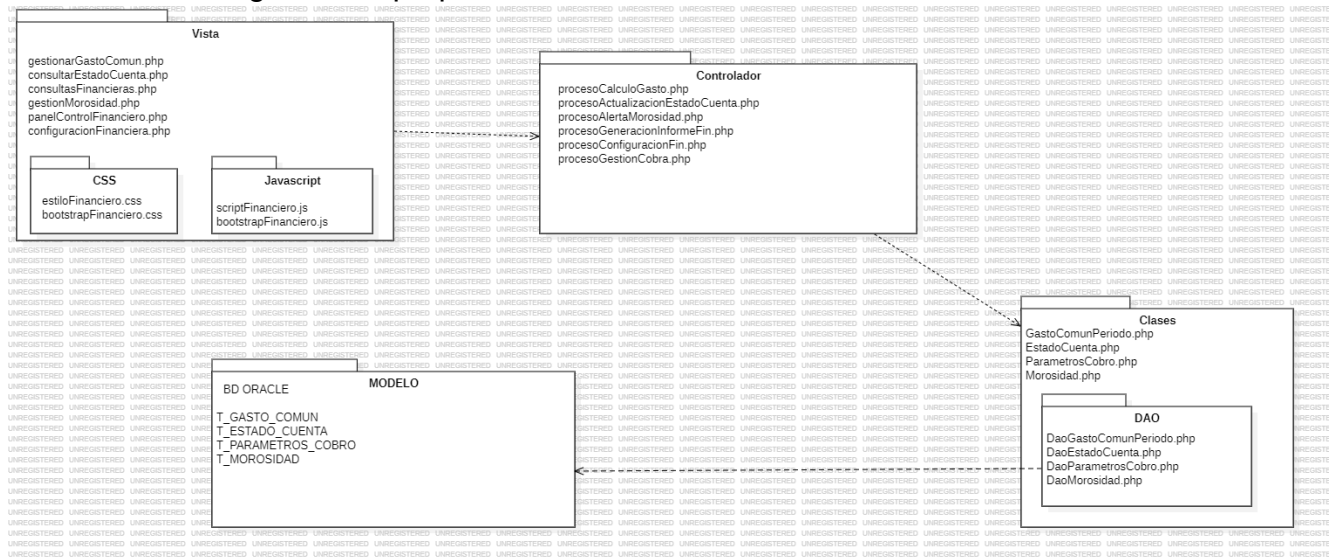
El componente **Cálculo de Gasto Común** permite generar el gasto común del edificio (RF-08), el cual luego es distribuido mediante el componente **Prorratio por Propiedad** (RF-09).

Posteriormente, el componente **Estado de Cuenta por Propiedad** permite visualizar la información financiera de cada unidad (RF-12).

Además, el sistema incluye el **Módulo de Morosidad**, encargado de gestionar deudas e intereses asociados a pagos atrasados (RF-13), considerando las reglas definidas en **Parametrización de Cobros** (RF-28).

El sistema también incorpora un **Módulo de Conciliación Bancaria**, que permite validar los pagos registrados, junto con un **Registro de Inconsistencias** para almacenar posibles errores o diferencias detectadas. Finalmente, el **Gestor de Notificaciones** permite informar a los usuarios sobre estados de cuenta, pagos o situaciones de morosidad.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes del Subsistema 3 organiza el sistema usando el patrón MVC (Modelo, Vista y Controlador).

- **Vista**: interfaz en PHP donde los usuarios consultan gastos comunes y estado de cuenta.
- **Controlador**: procesa las acciones del usuario, como generar gastos comunes, calcular prorratesos y detectar morosidad.
- **Clases y DAO**: contienen la lógica del sistema y gestionan el acceso a la base de datos.
- **Modelo (Base de datos)**: almacena la información de gastos, cuentas y reglas de cobro en Oracle.

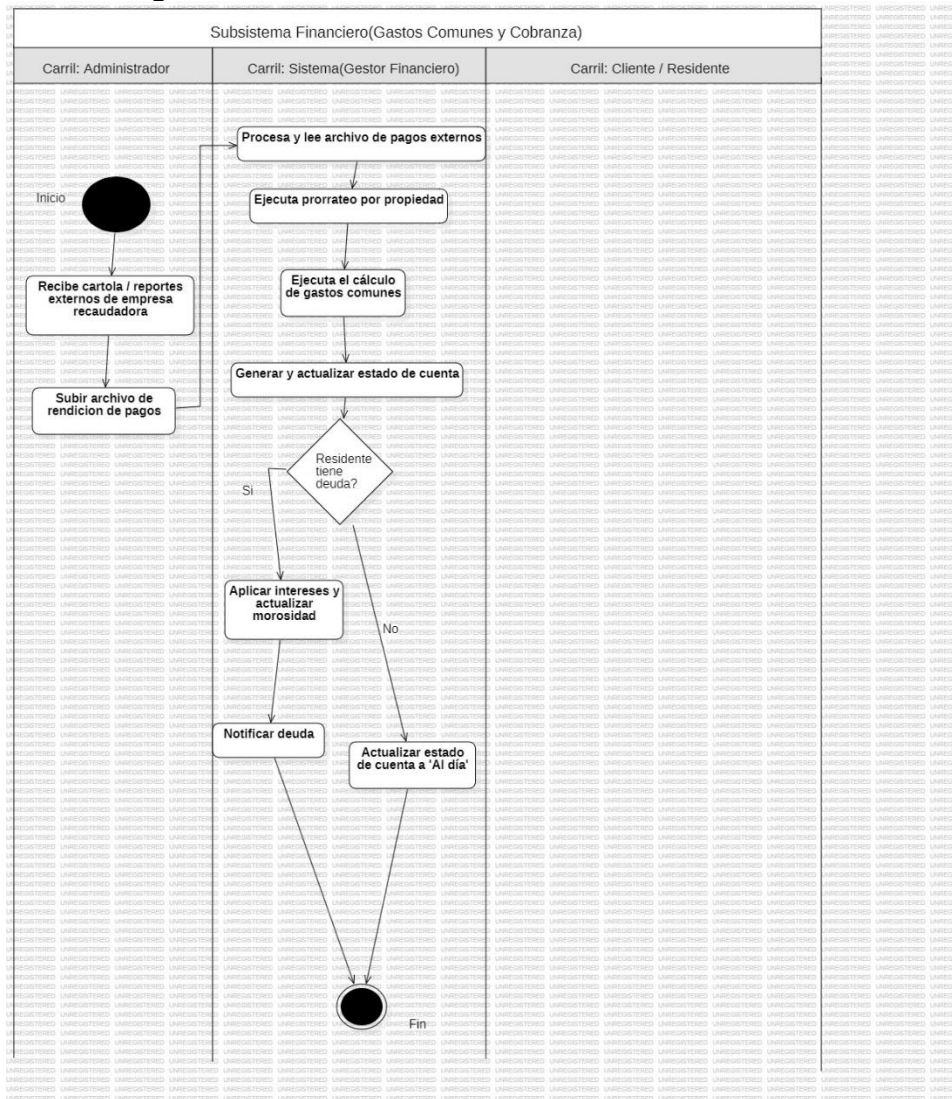
4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de esta vista es representar el flujo de trabajo dinámico del **Subsistema 3: Gestión Financiera (Gastos Comunes y Cobranza)**, mostrando cómo se ejecutan los procesos asociados a la generación de gastos comunes, prorrateso por propiedad, consulta de estados de cuenta y gestión de morosidad.

Permite visualizar los pasos que sigue el sistema desde el cálculo del gasto común del período, su distribución entre las propiedades, la generación de estados de cuenta y la aplicación de reglas de cobro e intereses

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama de actividad del **Subsistema Financiero (Gastos Comunes y Cobranza)** representa el flujo del proceso de cálculo y gestión de gastos comunes mediante tres carriles de responsabilidad:

- **Administrador:** recibe y revisa los reportes externos de la empresa recaudadora y carga el archivo de rendición de pagos en el sistema.
- **Sistema (Gestor Financiero):** procesa el archivo de pagos, ejecuta el cálculo del gasto común del período (RF-08), realiza el prorrateo por propiedad (RF-09) y genera o actualiza el estado de cuenta de cada residente (RF-12). Además, verifica si existen deudas y, en caso afirmativo, aplica intereses y registra la morosidad según las reglas configuradas (RF-13 y RF-28).
- **Cliente / Residente:** recibe la notificación del estado de su cuenta actualizado, pudiendo quedar **al día o con deuda registrada** en el sistema.

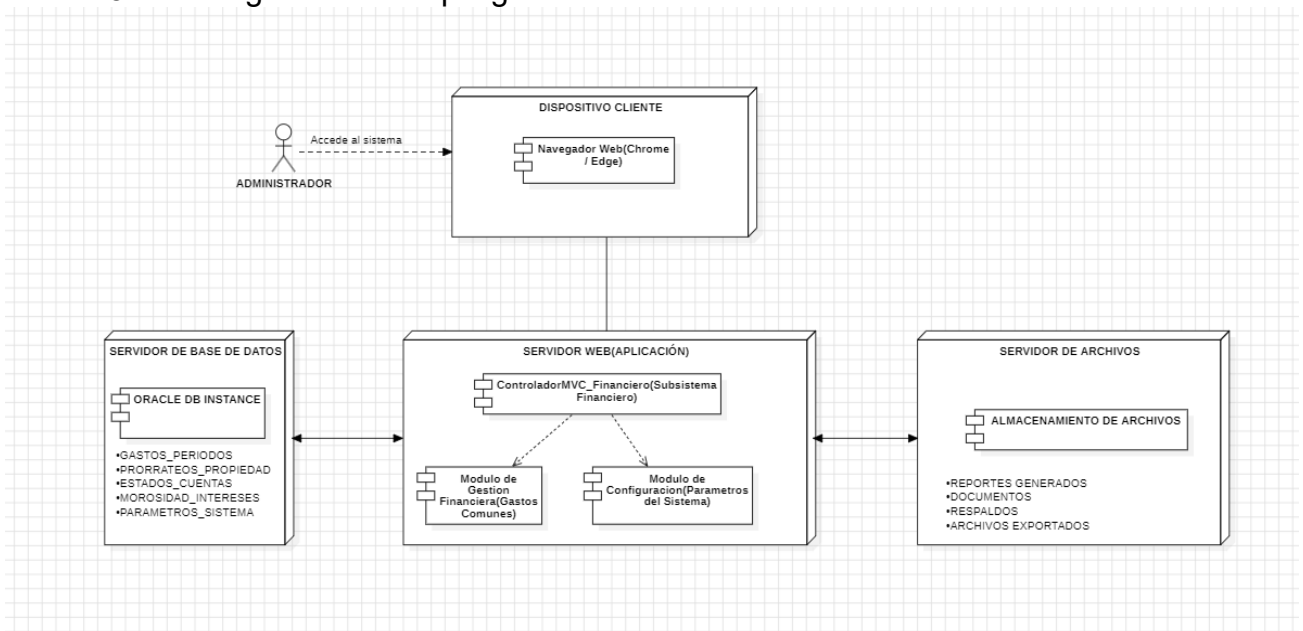
De esta forma, el diagrama muestra cómo el sistema procesa la información financiera y actualiza los estados de cuenta de las propiedades dentro del proceso de gestión de gastos comunes y cobranza.

4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de este diagrama es representar la arquitectura física y la infraestructura tecnológica del sistema. Detalla la topología de la red mediante la distribución de los componentes de software en servidores independientes: un servidor web encargado de ejecutar la lógica en PHP (MVC) y un servidor aislado que aloja la base de datos Oracle (DAO). Además, describe el uso de protocolos seguros de comunicación como HTTPS y TCP/IP para la conexión con el usuario y la pasarela Webpay, garantizando un entorno escalable, eficiente y protegido contra vulnerabilidades.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

Paso 1: Recopilar la información: El sistema lee un archivo externo con los datos bancarios para saber exactamente qué reglas y montos debe aplicar en los cobros del mes.

Paso 2: Calcular y dividir: Con esa información, el software calcula el gasto común total, lo divide matemáticamente por cada departamento (prorrateo) y genera el estado de cuenta final para cada propiedad.

Paso 3: Revisar que el dinero cuadre (Conciliación): El Administrador usa el sistema para validar los pagos de forma manual. Al mismo tiempo, el software hace una revisión automática con el banco; si encuentra alguna diferencia o error, lo deja anotado en un registro para que no se pierda.

Paso 4: Avisar a los residentes: Finalmente, el sistema revisa los estados de cuenta para detectar quiénes están atrasados en sus pagos (morosidad) y se encarga de enviarle automáticamente una notificación de cobro al Co-propietario o Residente.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito de esta sección es definir los atributos de calidad que deben cumplir las funciones del **Subsistema 3: Gestión de Gastos Comunes y Cobranza**. Estos atributos aseguran que los procesos de cálculo del gasto común ,prorratio por propiedad , generación de estados de cuenta, detección de morosidad e intereses y parametrización de reglas de cobro se ejecuten de forma correcta, segura y eficiente.

De esta manera, el sistema garantiza precisión en los cálculos financieros, confiabilidad en la información de cada propiedad y transparencia en la administración de los gastos comunes del edificio.

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN
Usabilidad	Capacidad del sistema para permitir que administradores y residentes consulten estados de cuenta, gastos comunes y pagos de manera clara e intuitiva.	El subsistema debe facilitar la consulta de estados de cuenta por propiedad (RF-12) y permitir comprender fácilmente los cobros, intereses o deudas asociadas a cada departamento.
Rendimiento	Capacidad del sistema para ejecutar procesos financieros y responder a las consultas de los usuarios en tiempos adecuados.	Los procesos de generación de gasto común (RF-08) y prorratio por propiedad (RF-09) deben ejecutarse de forma eficiente para entregar resultados rápidos y mantener actualizado el estado financiero del edificio.
Fiabilidad	Capacidad del sistema para ejecutar cálculos financieros de forma precisa y consistente.	El sistema debe garantizar exactitud en el cálculo de prorratios, estados de cuenta y morosidad (RF-09, RF-12, RF-13) evitando errores en los montos cobrados a los residentes.

Seguridad	Capacidad del sistema para proteger la información financiera y restringir el acceso según el rol del usuario.	El subsistema gestiona información sensible como deudas, pagos y reglas de cobro , por lo que debe asegurar el acceso solo a usuarios autorizados y proteger los datos almacenados.
Mantenibilidad	Facilidad para modificar o actualizar reglas de cobro, intereses o cálculos financieros sin afectar el funcionamiento general del sistema.	La parametrización de reglas de cobro e intereses (RF-28) permite adaptar el sistema a cambios administrativos sin necesidad de rediseñar la aplicación.

5.3. Reglas y criterios de evaluación de calidad

Para comprobar de manera objetiva que el Subsistema 3 (Gestión de Gastos Comunes y Cobranza) cumple con los atributos de calidad definidos, se aplicarán las siguientes reglas, métricas y métodos de evaluación sobre los procesos de generación de gastos, prorrateo, consulta de estados de cuenta y gestión de morosidad.

Métricas y Criterios de Aceptación:

- **Rendimiento:**

El sistema debe ejecutar el proceso de generación del gasto común del período (RF-08) y el prorrateo por propiedad (RF-09) en un tiempo máximo de 5 segundos bajo condiciones normales. Además, la consulta del estado de cuenta por propiedad (RF-12) por parte de los residentes no debe superar 2 segundos de tiempo de respuesta.

- **Usabilidad:**

Los usuarios deben poder consultar su estado de cuenta, visualizar deudas y entender los cargos aplicados sin dificultad. En las pruebas de uso, al menos el 90% de los usuarios debe ser capaz de consultar su estado de cuenta o identificar su deuda sin errores de navegación.

- **Fiabilidad:**

El sistema debe garantizar precisión en todos los cálculos financieros relacionados con el prorrateo de gastos comunes (RF-09) y el cálculo de morosidad e intereses (RF-13). La tolerancia al error matemático será 0%, asegurando que los montos calculados coincidan exactamente con los valores definidos en las reglas del sistema.

- **Seguridad:**

El acceso a la información financiera debe estar controlado por roles de usuario. Los residentes solo podrán consultar su propio estado de cuenta (RF-12), mientras que las funciones de configuración de cobros e intereses (RF-28) estarán restringidas únicamente al administrador del sistema.

- **Pruebas de Carga y Rendimiento:**

Se utilizarán herramientas de simulación de carga para evaluar el comportamiento

del sistema cuando múltiples residentes consulten simultáneamente sus estados de cuenta o cuando el administrador ejecute procesos masivos como la generación del gasto común.

• **Pruebas de Usabilidad:**

Se realizarán evaluaciones de la interfaz del sistema con usuarios representativos (administradores y residentes) para verificar que la consulta de gastos comunes y estados de cuenta sea clara y comprensible.

• **Pruebas Unitarias del Cálculo Financiero:**

Se implementarán pruebas automatizadas para validar que los algoritmos de **prorratio de gastos comunes (RF-09)** y **cálculo de intereses por morosidad (RF-13)** generen resultados correctos según las reglas configuradas.

• **Validación de Reglas de Cobro:**

Se verificará que la **parametrización de reglas de cobro e intereses (RF-28)** se aplique correctamente en los cálculos financieros y en la generación de los estados de cuenta.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

El propósito de esta sección es detallar las directrices de ingeniería de software implementadas en la arquitectura del sistema. Estos principios guían la estructuración del código y los módulos con el fin de mitigar riesgos de regresión, asegurar un mantenimiento ágil frente a cambios regulatorios o administrativos de la copropiedad y asegurar la escalabilidad del sistema.

6.2. Principios de diseño

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones. Por ejemplo, el panel de "Gestión de Gastos" se enfoca únicamente en el ingreso físico de los cobros del periodo, derivando las alertas de cobros atrasados al módulo exclusivo de

		"Morosidad".
Acoplamiento	Grado de dependencia e interacción directa que existe entre los diferentes paneles o módulos del sistema.	Se busca un acoplamiento débil entre las pantallas del sistema. Esto permite que si la administración decide modificar visualmente el formulario de registro de un gasto, esta acción no altere ni rompa de forma directa la visualización del estado de cuenta final que recibe el residente en su propio portal.
Abstracción	Consiste en aislar los procesos complejos eliminando los detalles matemáticos o internos que no son relevantes para la operación diaria del usuario.	Toda la complejidad matemática detrás de la fórmula de distribución por alcúotas se oculta al usuario. El sistema expone un botón simple llamado "Calcular Prorratio", aislando al administrador de tener que calcular manualmente los porcentajes por cada departamento.
Encapsulamiento	Mecanismo que restringe y protege el acceso o modificación directa a la información sensible del sistema, permitiendo cambios solo a través de los flujos autorizados.	Los saldos de dinero y estados de deuda de los departamentos están protegidos. Ningún usuario externo puede modificarlos directamente a su voluntad; solo se actualizan automáticamente cuando el sistema valida físicamente el ingreso de un pago en la pantalla de recaudación.
Modularidad	Propiedad que permite dividir la plataforma en secciones independientes y autónomas para facilitar su operación y actualización.	El sistema financiero está dividido en bloques visuales independientes: un módulo para el Cálculo del Gasto Común, otro para el Estado de Cuenta por Propiedad y otro para las Notificaciones, permitiendo realizar mejoras en uno de ellos sin congelar la operación del resto de la plataforma.

7. PROTOTIPO

7.1. Propósito

El propósito fundamental de este prototipo es materializar la lógica del **Subsistema 3** en una interfaz funcional, intuitiva y robusta, permitiendo validar la automatización de la gestión contable de la comunidad.

En concreto, el prototipo busca:

Demostrar el funcionamiento técnico de los procesos críticos como el **cálculo y prorratio de gastos comunes** y la aplicación automatizada de **intereses por mora** (según RN-07), asegurando la precisión de los datos frente a los requerimientos RF-08, RF-09, RF-12, RF-13 y RF-28.

Garantizar la Integridad de la Información: Implementar y probar mecanismos de **prevención de errores financieros** (RN-08)

7.2. Mockups (imágenes con una breve descripción)
Módulo de Gestión de Gastos y Prorratio

Descripción: cumple directamente con los requerimientos **RF-08 (Generación de gasto común)** y **RF-09 (Generación de gasto común por propiedad)**, tal como se muestra

Gestión de Gastos

Gasto Común y Gastos Extraordinarios del periodo

Junio 2026

Gasto Común

\$923.000

5 ítems

Ext. Ejecutados

\$585.000

2 resueltos

Ext. En Curso / Est.

\$1.880.000

2 activos

Total General

\$3.388.000

Junio 2026

Gasto Común — Junio 2026

\$923.000

+ Agregar ítem

Concepto	Categoría	Proveedor	Comprobante	Monto	% del Total GC
MantenCIÓN ascensor	MantenCIÓN	TecnoElev Ltda.	BOL-001	\$180.000	19.5%
Servicio de aseo	Limpieza	CleanPro SpA	FAC-0234	\$220.000	23.8%
Guardia seguridad	Seguridad	Segur-Max	FAC-0891	\$350.000	37.9%
Agua potable	Agua	Aguas Andinas	BOL-4421	\$95.000	10.3%
Electricidad áreas comunes	Electricidad	Enel	BOL-7890	\$78.000	8.5%
SUBTOTAL GASTO COMÚN				\$923.000	

Gastos Extraordinarios

2 activos

+ Agregar gasto extra

Fuga de agua en tubería principal

urgente

Fuga de Agua

Prorratar

Sala de Máquinas - Reportado: 2026-05-12

Costo real \$420.000

Resuelto

Cortocircuito tablero eléctrico piso 3

urgente

Eléctrico

Piso 3 - Reportado: 2026-04-28

Costo real \$165.000

Resuelto

Grieta en muro fachada norte

alta

Estructural

Prorratar

Fachada Norte - Reportado: 2026-06-01

Estimado \$1.200.000

Cotizando

Falla sistema de ventilación

alta

Ventilación / Gas

Prorratar

Estacionamiento B-1 - Reportado: 2026-06-05

Estimado \$680.000

En Curso

Ejecutados: \$585.000

Estimados activos: \$1.880.000

Subtotal ext.: \$2.465.000

Total General — Junio 2026

Gasto Común \$923.000 + Extraordinarios ejecutados \$585.000 = \$1.508.000 (+ \$1.880.000 estimados pendientes)

Calcular Prorratio

Descripción: Esta interfaz, es fundamental para garantizar la **transparencia financiera** del sistema, dando cumplimiento directo al **RF-12 (Estado de cuenta por propiedad)**.

Funcionalidad: Permite al residente y al administrador consultar en tiempo real el historial detallado de movimientos, incluyendo cargos, pagos efectuados y el saldo actual consolidado.

Perfil Administración

Estado de Cuenta

RF-12 — Historial y estado vigente por propiedad

Propiedades

101

Carlos Mendoza

\$0

Al Día

Unidad 101 — Departamento

Carlos Mendoza · RUT 10.111.222-3

c.m@email.com · +56 9 1111 2222

Total Cargos

\$0

Total Pagos

+\$0

Saldo Actual

\$0

Exportar PDF

Perfil propietario

Estado de Cuenta

RF-12 — Historial y estado vigente por propiedad

Mis Propiedades

101

Carlos Mendoza

\$0

Al Día

EST01

Carlos Mendoza

\$0

Al Día

Unidad 101 — Departamento

Carlos Mendoza · RUT 10.111.222-3

c.m@email.com · +56 9 1111 2222

Total Cargos

\$0

Total Pagos

+\$0

Saldo Actual

\$0

Exportar PDF

Historial de Movimientos

Panel de Control de Morosidad

Descripción: constituye la herramienta de análisis crítico del Subsistema Financiero para dar cumplimiento al RF-13 (Gestión de morosidad).

Funcionalidad: El sistema identifica automáticamente a los residentes morosos, clasificando el nivel de deuda (Leve, Moderado, Crítico) basándose en la antigüedad. Incluye un botón para el cálculo automatizado de intereses (tasa del 1.5% mensual) y multas aplicadas.

Gestión de Morosidad

RF-13 — Identificación, clasificación e intereses automáticos

Calcular Intereses

Unidades Morosas 3 de 10 propiedades	Deuda Total \$1.029.680 incluye intereses y multas	Intereses Acumulados \$67.680 tasa 1.5% mensual	Alertas Enviadas 8 este mes
---	---	--	--

Módulo de Parametrización del Sistema

Descripción: Estas interfaces, detalladas en las imágenes, operan bajo el requerimiento **RF-28 (Parametrización del sistema)**

Permite gestionar los diversos ítems que componen el gasto común (fijos, variables o por porcentaje) y definir los tipos de propiedad y sus coeficientes de prorroateo. Esta funcionalidad garantiza la **escalabilidad** del sistema

Parametrización del Sistema

RF-28 — Configuración de conceptos, reglas, intereses y plantillas

Guardar Cambios

\$ Conceptos Mora Notificaciones Comunidad

Conceptos de Cobro

+ Nuevo Concepto

Concepto	Tipo	Valor	Aplica a	Activo
Gasto Común Base	Variable	Prorroateo	Todos	☒
Fondo de Reserva	Porcentaje	10%	Todos	☒
Seguro Edificio	Fijo	\$15.000	Departamento	☒
Mantenimiento Ascensor	Fijo	\$5.000	Departamento	☒
Estacionamiento Extra	Fijo	\$25.000	Estacionamiento	☐

Tipos de Propiedad

Departamento

Coeficiente base: 9%



Local Comercial

Coeficiente base: 22%



Estacionamiento

Coeficiente base: 3.5%



Bodega

Coeficiente base: 15%



Día de vencimiento (cada mes)

5

Los cobros vencen el día 5 de cada mes

Días de gracia

5

No se aplican intereses hasta el día 10

Aplicar intereses por mora
Calcular automáticamente intereses en cada cierre de mes

Tasa de interés mensual (%)

1,5

% mensual sobre deuda vencida

Interés compuesto
Los intereses se acumulan sobre el total (deuda + intereses previos)

Ejemplo de cálculo:
Deuda base de \$100.000 con 1.5% mensual: Mes 1: \$101.500 → Mes 2: \$103.000 → Mes 3: \$104.500

7.3. Justificar herramientas de prototipado

Para el desarrollo del **Subsistema Financiero (Subsistema 3)**, se seleccionó **Figma** como herramienta principal de prototipado por su capacidad para cerrar la brecha entre el diseño conceptual y la implementación técnica, asegurando que el producto final responda fielmente a los requerimientos.

Justificación respecto a los Requerimientos

RF-12 (Estado de cuenta) y RF-13 (Gestión de morosidad): Figma permitió visualizar la jerarquía visual de estos módulos, asegurando que los indicadores de deuda y el botón de "Calcular Intereses" estuvieran ubicados estratégicamente para no generar errores.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

El propósito de esta sección es evaluar la calidad de la interfaz y la experiencia de usuario (UX) del prototipo del **Sistema de Gestión de Copropiedad / Condominios** (centrado en módulos como Gastos Comunes, Prorratio, Control de Morosidad e Inicio de Sesión) mediante la metodología de **Evaluación Heurística de Jakob Nielsen**.

A través de esta inspección analítica, se persiguen los siguientes objetivos específicos basados en los hallazgos del archivo `Evaluacion_Heuristica_Nielsen.xlsx`:

- **Identificar violaciones de usabilidad:** Detectar y registrar de forma sistemática aquellos componentes, flujos o mensajes de la interfaz que transgreden los 10 principios heurísticos de Nielsen (tales como la falta de visibilidad del estado del sistema o la ausencia de prevención de errores).
- **Cuantificar la gravedad de los problemas:** Clasificar cada vulnerabilidad encontrada utilizando la escala de severidad de Nielsen (de 0 a 4), con el fin de priorizar los esfuerzos de desarrollo en los fallos más críticos (por ejemplo, el procesamiento inmediato e irreversible del botón masivo de cobros, catalogado con severidad 3).
- **Establecer métricas base de calidad:** Consolidar un diagnóstico cuantitativo inicial del prototipo (el cual registra actualmente un total de **7 problemas de usabilidad** con un **promedio de severidad de 1.71**) para medir el impacto de futuras iteraciones.
- **Proponer recomendaciones de optimización:** Proveer soluciones de rediseño y mejoras técnicas específicas para cada hallazgo (como la incorporación de ventanas de confirmación, validación de datos en tiempo real y alertas visuales focalizadas), garantizando que el sistema final sea intuitivo, eficiente y seguro para el usuario administrador.

8.2. Lista de verificación

ID	Heurística de Nielsen	Descripción del Problema Detectado Resumen_y_Referencia_Heuristica_Nielsen_Corregido	Ubicación en el Prototipo Resumen_y_Referencia_Heuristica_Nielsen_Corregido	Severidad (0-4) Resumen_y_Referencia_Heuristica_Nielsen_Corregido	Recomendación de Rediseño / Solución Técnica
01	H3: Control y libertad del usuario	El botón " <i>Confirmar y Emitir Cobros</i> " procesa y publica las deudas de forma inmediata para las 87 unidades del condominio con un solo clic. No existe un botón para cancelar ni una ventana flotante que permita dar marcha atrás si se presionó por error.	Módulo de Gastos Comunes / Panel de Gastos	3 (Mayor)	Implementar un cuadro de diálogo modal intermedio de confirmación (" <i>¿Está seguro de emitir los cobros para las 87 unidades?</i> ") y un botón de "Cancelar" o un temporizador de cuenta regresiva de 5

					segundo con opción "Desahacer".
02	H5: Prevención de errores	El formulario visual para añadir un gasto mensual permite escribir números negativos o dejar el campo de dinero completamente vacío. El botón para guardar no se bloquea ni advierte el error antes de ser pulsado.	Ventana emergente "Añadir Gasto" en el módulo de Gastos Comunes	2 (Menor)	Aplicar validación de formularios mediante expresiones regulares (Regex) del lado del cliente. Deshabilitar el botón "Guardar" (estado disabled) si el campo está vacío o contiene valores \$\\le 0\$.
03	H1: Visibilidad	Al registrar un pago o enviar una solicitud de vinculación de	Módulos de Liquidación de Propiedad y Mis	1 (Cosmético)	Integrar un comp

	del estado del sistema	departamento, la ventana se cierra de golpe o altera la tabla, pero omite un mensaje flotante explícito de éxito que ratifique visualmente la transacción.	Propiedades		ponente de notificación reactiva ("Toast notification") en la esquina superior derecha que muestre un mensaje claro con un ícono verde de éxito durante 3 segundos: "Pago registrado exitosamente".
04	H10: Ayuda y documentación	Las pantallas con lógicas de cálculo complejas (como la aplicación de la tasa de interés compuesto por morosidad o la tabla de coeficientes de copropiedad) no tienen textos de apoyo ni descripciones	Pantallas de Parametrización y Liquidaciones	2 (Menor)	Incorporar componentes de ayuda flotante (Tooltips) mediante

		explicativas.			ícono s de inform ación (i). Al posici onar el cursor sobre ellos, deben despl egar la fórmul a mate mátic a exact a y un ejemp lo breve de cálcul o del prorra teo.
0 5	H4: Cons isten cia y están dare s	El botón de ingreso de la interfaz inicial (Login) maneja una estética de colores y transparencias fijas que desentona con los componentes visuales y botones estándar del resto de los menús internos.	Pantalla de Inicio de Sesión (Login)	1 (Cosmético)	Homo logar los estilo s en las hojas de estilo (CSS). Modifi car el botón del Login aplica ndo la mism a paleta de

					colores corporativos, bordes redondeados (border-radius) y directrices de diseño o interactivo del resto de la plataforma.
06	H9: Reconocer, diagnosticar y recuperarse de errores	Al fallar en las credenciales de acceso, el sistema muestra una barra superior de advertencia muy clara, pero no tiñe en rojo el borde de la casilla exacta (correo o contraseña) que causó la equivocación.	Pantalla de Inicio de Sesión (Login)	1 (Cosmético)	Utilizar clases dinámicas para alternar estados de error en los inputs. Ante un fallo de autenticación, aplicar una clase input

					- error que cambie el borde de los campos de texto específicos a color rojo (#FF0000) y agregue un microtexto descriptivo debajo del campo.
07	H7: Flexibilidad y eficiencia de uso	La gestión de morosidad carece de una herramienta de selección masiva para enviar recordatorios a varios residentes al mismo tiempo, obligando al administrador a abrir las fichas de los deudores una por una.	Panel de Morosidad	2 (Menor)	Añadir una columna de casillas de verificación (<i>Checkboxes</i>) en la tabla del panel de deudores, junto con un control maestro

					ro en la cabec era para "Sele ccion ar todo" y un botón de acció n masiv a superi or: "Envi ar recor datori o a selec ciona dos".
--	--	--	--	--	--

8.3. Análisis y métricas de resultados

A partir de la inspección realizada en la lista de verificación, se procedió a efectuar un análisis cuantitativo y cualitativo de la usabilidad del prototipo. Este análisis permite establecer una línea base de calidad de la interfaz antes de iniciar la fase de construcción del código definitivo.

8.3.1. Indicadores Globales de Usabilidad

Para cuantificar el estado de la experiencia de usuario (UX) en el *Software Edificio Mirador*, se consolidaron las siguientes métricas clave de la evaluación:

- **Total de problemas de usabilidad detectados:** 7 problemas en total.
- **Promedio general de severidad: 1.71** (en la escala de 0 a 4 de Nielsen). Al situarse bajo el umbral de 2.0, indica que la plataforma posee una base sólida y limpia, donde la mayoría de los hallazgos corresponden a mejoras cosméticas o menores que no bloquean la operación global.
- **Problemas Críticos/Catastróficos (Severidad 4):** 0. No se encontraron fallos críticos que causen la pérdida total de control del sistema o caídas de la infraestructura.
- **Problemas Mayores (Severidad 3):** 1 (14.3% del total). Correspondiente al botón de emisión inmediata de cobros masivos sin confirmación en el módulo de gastos comunes.
- **Problemas Menores (Severidad 2):** 3 (42.9% del total). Concentrados en la rigidez de flujos administrativos y la falta de validación temprana en formularios.

- **Problemas Cosméticos (Severidad 1):** 3 (42.9% del total). Fallos visuales leves que afectan la consistencia estética o la precisión de alertas en pantalla.

8.3.2. Tabla de Distribución y Madurez por Heurística

Para identificar qué áreas del diseño requieren mayor atención de ingeniería de software, se estructuró la siguiente matriz de distribución, la cual mide el impacto real de los problemas por cada principio de Nielsen:

Heurística Evaluada	Problemas Detectados	Severidad Máxima	Severidad Promedio	Estado de la Heurística	Prioridad de Intervención
H1: Visibilidad del estado del sistema	1	1	1.0	Faltan mensajes de confirmación de éxito ("Toasts").	3 - Baja
H2: Relación entre sistema y mundo real	0	0	0.0	Cumple plenamente. Vocabulario adecuado para administración.	Sin problemas
H3: Control y libertad del usuario	1	3	3.0	Riesgo Operativo Mayor. Acción crítica irreversible.	1 - Alta
H4: Consistencia y estándares	1	1	1.0	Estética del botón de Login desalineada de la guía interna.	3 - Baja
H5: Prevención de errores	1	2	2.0	Formulario de gastos permite datos vacíos o negativos.	2 - Media
H6: Reconocimiento antes que recuerdo	0	0	0.0	Cumple plenamente. Información visual persistente.	Sin problemas
H7: Flexibilidad y eficiencia de uso	1	2	2.0	Flujo ineficiente. Falta	2 - Media

				selección masiva en morosidad.	
H8: Estética y diseño minimalista	0	0	0.0	Cumple plenamente. Interfaces limpias y sin sobrecarga.	Sin problemas
H9: Reconocer y recuperarse de errores	1	1	1.0	Alerta de error clara, pero no resalta la casilla exacta.	3 - Baja
H10: Ayuda y documentación	1	2	2.0	Falta de glosario o "Tooltips" en lógicas de cálculo complejas.	2 - Media

8.3.3. Análisis Cualitativo y Diagnóstico del Sistema

El diagnóstico demuestra que el prototipo destaca positivamente en la estructura de navegación, la limpieza visual y el uso de un lenguaje técnico natural para el rubro inmobiliario (Heurísticas H2, H6 y H8 con cero incidencias). Esto valida que la definición inicial de los requerimientos funcionales fue bien interpretada en la etapa de diseño de interfaces.

Sin embargo, desde la perspectiva de la ingeniería de software, se observa una vulnerabilidad importante en la robustez de los procesos contables. La falta de un mecanismo de "Deshacer" o de una ventana de confirmación antes de publicar deudas a gran escala (**Heurística H3, Severidad 3**) constituye un riesgo crítico. En un entorno real con 87 departamentos en producción, un clic accidental del administrador generaría desconfianza y confusión inmediata en la comunidad de copropietarios, lo que atenta directamente contra los objetivos estratégicos definidos para este sistema.

De igual forma, la presencia de problemas de severidad 2 en las heurísticas de **Prevención de errores (H5)** y **Eficiencia de uso (H7)** revela la necesidad de fortalecer las validaciones en los formularios de entrada de datos (impidiendo montos ilógicos o negativos) e incorporar herramientas de automatización masiva. Esto evitará que la administración sufra cuellos de botella operativos al verse obligada a procesar los recordatorios de cobranza de manera individual.

8.3.4. Plan de Mitigación y Priorización (Backlog Técnico)

Para asegurar el cumplimiento de los atributos de calidad de *Fiabilidad* y *Usabilidad* definidos en la sección 5 del DAS, las correcciones se priorizarán en el plan de desarrollo de la siguiente manera:

1. **Prioridad 1 (Crítica - Sprint Inmediato):** Corrección del problema de control y libertad (ID 01). Es un requisito obligatorio añadir el modal de confirmación antes de la ejecución de código en el módulo financiero.

2. **Prioridad 2 (Media - Sprints de Estabilización):** Implementación de validaciones del lado del cliente mediante JavaScript en el formulario de gastos (ID 02), adición de herramientas de selección masiva en la tabla de deudores (ID 07) y desarrollo de elementos de ayuda contextual (*Tooltips*) para los cálculos financieros (ID 04).
3. **Prioridad 3 (Baja - Pulido Estético):** Ajustes en las hojas de estilo CSS para homologar la apariencia del botón de login (ID 05), inyección dinámica de clases de error en los campos de texto (ID 06) y activación del componente reactivo de alertas de éxito (ID 03).

9. CONTROL DE VERSIONES

9.1. Propósito

El propósito de esta sección es establecer el mecanismo formal para registrar, organizar y hacer un seguimiento de todas las modificaciones realizadas en los artefactos del proyecto (como este Documento de Arquitectura, las hojas de cálculo de calidad y los archivos de diseño visual). Esto garantiza que el equipo trabaje siempre sobre la información más reciente, evitando la pérdida de avances o la duplicación de tareas.

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial))

El proyecto utiliza un esquema de **Versionamiento Semántico**, el cual clasifica la importancia de cada cambio bajo la estructura numérica X.Y.Z (por ejemplo, la versión actual de este documento es la 1.3).

- **X (Cambio Mayor):** Se incrementa cuando hay una reestructuración total del diseño o un cambio drástico en las reglas del negocio (por ejemplo, si se decide cambiar por completo el flujo de cobranza).
- **Y (Cambio Menor):** Se modifica cuando se añaden nuevas secciones o pantallas funcionales que complementan lo existente (como la adición de la sección de principios de diseño).
- **Z (Ajuste/Parche):** Se actualiza al realizar correcciones menores, cambios de formato o corrección de textos.

Justificación: Se prefiere este método sobre el secuencial simple o por fechas porque permite que cualquier integrante del equipo o revisor externo sepa instantáneamente el nivel de impacto y la madurez del diseño con solo mirar el número de la versión, manteniendo coherencia con el historial de revisiones del inicio del documento.

9.3. Justificar herramientas de versionamiento

Para la gestión del trabajo colaborativo se justifica el uso de las siguientes

herramientas institucionales:

1. **Git (Local):** Utilizado como el motor para capturar el historial de cambios de los archivos de diseño y documentación en las estaciones de trabajo de cada integrante. Permite crear "ramas" de trabajo separadas para que un miembro pueda evaluar la usabilidad en un archivo sin alterar el avance del compañero que redacta la arquitectura.
2. **GitHub (Nube):** Se utiliza como el repositorio centralizado donde se resguarda el proyecto. Su uso es indispensable para realizar revisiones cruzadas antes de consolidar una versión final del diseño, asegurando que todos los entregables estén respaldados de forma segura y accesible para el equipo.

10. CONCLUSIONES

El desarrollo y documentación de la arquitectura de software para el *Software Edificio Mirador* permite consolidar las siguientes conclusiones técnicas y estratégicas en el ámbito de la ingeniería de sistemas:

1. **Validación de la Arquitectura frente al Negocio:** El diseño arquitectónico basado en Capas y el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**, complementado con componentes **DAO**, demostró ser la solución idónea para resolver la problemática contable de la copropiedad. Esta estructura no solo garantiza la independencia de las interfaces web en PHP respecto a la persistencia en Oracle, sino que dota al sistema de la escalabilidad necesaria para procesar algoritmos complejos de prorrateo por propiedad y cálculo automatizado de morosidad sin degradar el rendimiento global del software.
2. **Mitigación de Riesgos Operativos mediante Calidad Heurística:** La integración de la evaluación heurística de Nielsen dentro del ciclo de diseño arquitectónico constituyó un paso crítico para salvaguardar la integridad financiera del proyecto. La detección temprana de vulnerabilidades de severidad mayor —específicamente el riesgo operativo de emitir cobros masivos sin confirmación previa (Heurística H3)— permitió reestructurar el *Backlog Técnico* antes de la escritura de código. Al aplicar las medidas de mitigación diseñadas (ventanas modales intermedias y validaciones del lado del cliente), se asegura un sistema final altamente fiable, preciso y blindado contra errores humanos involuntarios.
3. **Aseguramiento de Atributos de Calidad:** La formalización de métricas y criterios de aceptación específicos para los atributos de *Rendimiento*, *Fiabilidad*, *Seguridad* y *Usabilidad* define un marco de gobernanza claro para el equipo de desarrollo. Garantizar tiempos de respuesta inferiores a los 2 segundos en la consulta de estados de cuenta y una tolerancia del 0% en el error matemático del motor de prorrateo, posiciona a la plataforma como un sistema robusto capaz de recuperar la confianza de los residentes y agilizar sustancialmente los flujos financieros de la administración.

11. BIBLIOGRAFÍA

- **Bass, L., Clements, P., & Kazman, R.** (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Referencia para la implementación de los patrones MVC y DAO adoptados en el sistema).
- **Nielsen, J.** (1994). *Enhancing the explanatory power of usability heuristics*. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 152–158. (Referencia para la metodología de evaluación de usabilidad aplicada en la sección 8).
- **Oracle Corporation.** (2026). *Oracle Database Documentation Library*. Recuperado de la documentación oficial para la integración del modelo relacional de persistencia.
- **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

12. ANEXOS

Esta sección recopila el material complementario e instrumental que respalda las decisiones de diseño arquitectónico, la planificación temporal y los artefactos técnicos desarrollados para el *Software Edificio Mirador*.

12.1 Carta Gantt

La planificación temporal del proyecto se estructuró bajo una metodología híbrida, utilizando el marco de trabajo **Scrum** para el desarrollo de los Sprints y una **Carta Gantt** para fijar los hitos macro y los plazos de entrega institucionales. El proyecto contempla una duración total de 10 semanas, distribuidas desde la concepción inicial y el modelado arquitectónico hasta las pruebas de carga y el despliegue final en producción.

A continuación, se detalla la matriz de asignación de tiempos, fases y estados de los entregables del software:

ID	Fase / Actividad Técnica	Semanas de Ejecución	Horas Estimadas	Entregable Asociado	Estado Actual
01	Fase 1: Levantamiento de requerimientos, definición del ámbito y casos de uso.	Semanas 1 - 2	24 hrs	Documento de Especificación de Requerimientos (SRS).	Completado
02	Fase 2: Diseño de Vistas Arquitectónicas del Modelo 4+1 (Escenario,	Semanas 2 - 3	36 hrs	Diagramas UML e Historial de Revisiones del DAS.	Completado

	Lógica, Procesos, Desarrollo y Física).				
03	Fase 3: Prototipado de Interfaces de Alta Fidelidad en Figma (Módulos de Gastos Comunes, Estados de Cuenta, Morosidad y Configuración).	Semanas 3 - 4	30 hrs	Enlace al prototipo interactivo y Mockups del sistema.	Completado
04	Fase 4: Evaluación de Calidad Heurística de Nielsen y Análisis Cuantitativo de Severidad.	Semana 4	16 hrs	Planilla Evaluacion_Heuristica_Nielsen.xlsx y Plan de Mitigación.	Completado
05	Fase 5: Diseño de la Base de Datos Relacional y creación del Modelo Físico en Oracle.	Semana 5	20 hrs	Script SQL de creación de tablas (DDL) y restricciones DAO.	En Desarrollo
06	Sprint 1: Construcción de la Capa de Presentación (Vistas PHP/CSS) y Módulo de Autenticación (Login).	Semana 6	32 hrs	Código fuente del Login homologado y seguro.	Planificado
07	Sprint 2: Programación del Motor Financiero Principal (Algoritmos de	Semanas 7 - 8	45 hrs	Componente <i>Cálculo Gasto Común</i> e Interfaz de Conciliación.	Planificado

	Prorratio, Gastos y Carga de Cartola Bancaria).				
0 8	Sprint 3: Implementación del Módulo de Morosidad, Cálculo de Interés Compuesto y Gestor de Notificaciones por Email.	Semanas 8 - 9	40 hrs	Automatización de alertas y cálculo automatizado de deudas.	Planificado
0 9	Fase 6: Pruebas Unitarias de Cálculos Financieros y Pruebas de Carga/Rendimiento de Consultas Simultáneas.	Semana 10	18 hrs	Matriz de Casos de Prueba con Tolerancia 0% al error matemático.	Planificado
1 0	Fase 7: Despliegue en Servidor de Producción, Capacitación de la Administración y Cierre del Proyecto.	Semana 10	12 hrs	Manual de Usuario e Instalación, Acta de Aceptación del Software.	Planificado